



JAVASCRIPT MASTER STUDY DOCUMENTATION

CHAPTER 02: VARIABLES, DATA TYPES & TYPE SYSTEM

TOTAL SECTIONS

55 Modules + Project

CURRICULUM PART

Part 01 — Fundamentals (2/3)

CORE MECHANICS

Primitive vs Reference,
Coercion

REFERENCE STANDARD

ECMAScript 2026 / V8
Runtime

JavaScript-এর Data কীভাবে কাজ করে

Chapter 1-এ আমরা JavaScript কী, কীভাবে run করে, variable কী এবং `let`, `const`, `var` সম্পর্কে basic ধারণা নিয়েছি। এখন আমরা JavaScript-এর সবচেয়ে গুরুত্বপূর্ণ foundation — **Data Types & Type System** পুঙ্খানুপুঙ্খভাবে শিখব। এই অধ্যায়টি আয়ত্ত করলে পরবর্তী অধ্যায়গুলোর Operators, Conditions, Functions, Arrays, Objects, DOM এবং API বোঝা অত্যন্ত সহজ হয়ে যাবে।

PART 01 — DATA & PRIMITIVE FOUNDATIONS (1 – 9)

1 What is Data?

English: Data is information that a program can store, process, and use.

বাংলা: Data হলো এমন information যেটা একটি program store, process বা ব্যবহার করতে পারে।

REAL-LIFE EXAMPLE

একজন ছাত্রের পরিচয়:

```
Name    → "Ripon"  
Age      → 22  
CGPA     → 3.75  
Student  → true
```

JAVASCRIPT

DATA TYPES IN VARIABLES

```
const name = "Ripon";  
const age  = 22;  
const cgpa = 3.75;  
const isStudent = true;
```

2 What is a Data Type?

English: A data type defines what kind of value a piece of data is and what operations can be performed on it.

বাংলা: Data Type বলে দেয় কোনো value কী ধরনের data এবং সেই data-এর সাথে কী ধরনের কাজ করা যায়।

"Hello" → String | 100 → Number | true → Boolean

3 JavaScript Data Types Taxonomy

জাভাস্ক্রিপ্ট ডেটা টাইপকে প্রধানত ২টি ভাগে ভাগ করা হয়:

```
Data Types
├── Primitive (7 Types)
│   ├── 1. String
│   ├── 2. Number
│   ├── 3. BigInt
│   ├── 4. Boolean
│   ├── 5. Undefined
│   ├── 6. Null
│   └── 7. Symbol
└── Non-Primitive / Reference
    └── 8. Object (Object, Array, Function, Date, Map, Set)
```

4 Primitive vs Non-Primitive

Primitive: JavaScript-এর basic ও immutable value type (যেমন: String, Number, Boolean)।

Non-Primitive: মূলত Object/Reference-based structured collection (যেমন: `{ name: "Ripon", age: 22 }`)।

```
Primitive      → Single / basic value
Non-Primitive → Collection / structured complex value
```

5 String

English: A String is a sequence of characters used to represent text.

বাংলা: String হলো text বা character-এর ধারাবাহিক sequence।

```
const name = "Ripon";
```

6 String লেখার উপায়

জাভাস্ক্রিপ্টে স্ট্রিং ৩ ভাবে লেখা যায় (তিনটিই বৈধ):

JAVASCRIPT

QUOTES SYNTAX

```
const dQuotes = "Ripon"; // Double Quotes
const sQuotes = 'Ripon'; // Single Quotes
const bTicks  = `Ripon`; // Backticks (Template Literals)
```

7 String Example

JAVASCRIPT

STRING OUTPUT

```
const firstName = "Shariar";
const lastName = "Ahamed";

console.log(firstName);
console.log(lastName);
```

OUTPUT

```
Shariar
Ahamed
```

8 String Concatenation

+ অপারেটর দিয়ে একাধিক String যুক্ত করা যায়:

JAVASCRIPT

CONCATENATION

```
const firstName = "Shariar";
const lastName = "Ahamed";
const fullName = firstName + " " + lastName;
console.log(fullName);
```

OUTPUT

```
Shariar Ahamed
```

9 Template Literal

Modern JavaScript (ES6)-এ ব্যাকটিক ``` এবং `${variable}` দিয়ে সহজে স্ট্রিং ইন্টারপোলেশন করা যায়:

JAVASCRIPT

TEMPLATE LITERALS

```
const name = "Ripon";
const age = 22;
console.log(`My name is ${name} and I am ${age} years old.`);
```

OUTPUT

```
My name is Ripon and I am 22 years old.
```

PART 02 — NUMBERS, BOOLEANS & SPECIAL VALUES (10 – 27)

10 Number

জাভাস্ক্রিপ্টের `Number` টাইপ পূর্ণসংখ্যা (Integer), দশমিক (Decimal/Floating-point) এবং ঋণাত্মক (Negative) সব সংখ্যাকেই রিপ্রেজেন্ট করে:

JAVASCRIPT

NUMBER VARIANTS

```
const age = 22;           // Integer
const cgpa = 3.75;        // Decimal
const temperature = -5;   // Negative
```

11 Number Example

JAVASCRIPT

LOGGING NUMBERS

```
const age = 22;
const price = 499.99;
const temperature = -10;

console.log(age);
console.log(price);
console.log(temperature);
```

OUTPUT

```
22
499.99
-10
```

12 Checking Number Type

JAVASCRIPT

TYPEOF NUMBER

```
console.log(typeof 100); // number
console.log(typeof 3.14); // number
```

OUTPUT

```
number
number
```

13 Special Number Values

জাভাস্ক্রিপ্টের Number টাইপের মধ্যে ৩টি অত্যন্ত গুরুত্বপূর্ণ স্পেশাল ভ্যালু রয়েছে:

`Infinity` (অসীম ধনাত্মক), `-Infinity` (অসীম ঋণাত্মক), `NaN` (Not a Number)

14 Infinity

কোনো সংখ্যাকে 0 দিয়ে ভাগ করলে এরর না দিয়ে জাভাস্ক্রিপ্ট `Infinity` রিটার্ন করে:

```
console.log(10 / 0);
```

OUTPUT

```
Infinity
```

15 `NaN` (Not a Number)

গাণিতিকভাবে অর্থহীন অপারেশন করলে `NaN` তৈরি হয়:

JAVASCRIPT

NAN GENERATION

```
console.log("Hello" * 5);  
console.log(Number("abc"));
```

OUTPUT

```
NaN  
NaN
```

16 Important: `NaN` Type

⚠️ **চমকপ্রদ বিষয়:** নাম `Not a Number` হলেও এর টাইপ হলো `number` ! কারণ specification অনুযায়ী এটি `Number` টাইপের একটি `invalid numeric state` নির্দেশক।

```
console.log(typeof NaN);
```

OUTPUT

```
number
```

17 Boolean

বুলিয়ানের মাত্র দুটি মান: `true` অথবা `false`।

JAVASCRIPT

BOOLEAN

```
const isLoggedIn = true;  
const isAdmin = false;  
console.log(isLoggedIn);  
console.log(isAdmin);
```

OUTPUT

```
true  
false
```

18 Real-Life Boolean Example

ওয়েবসাইটে ইউজার লগইন স্ট্যাটাস চেক করে কন্ডিশনাল রেন্ডারিং:

JAVASCRIPT

DECISION FLOW

```
const isLoggedIn = true;  
if (isLoggedIn) {  
  console.log("Welcome back!");  
}
```

OUTPUT

```
Welcome back!
```

19 Undefined

একটি ভ্যারিয়েবল ডিক্লেয়ার করা হয়েছে কিন্তু কোনো ভ্যালু এসাইন করা হয়নি:

JAVASCRIPT

UNDEFINED STATE

```
let name;  
console.log(name);
```

OUTPUT

```
undefined
```

20 `undefined` Example

JAVASCRIPT

VALUE AND TYPE

```
let result;  
console.log(result);  
console.log(typeof result);
```

OUTPUT

```
undefined  
undefined
```

এখানে ভ্যালুও `undefined` , এবং টাইপও `undefined` ।

21 Null

Null: Intentionally empty / no value. প্রোগ্রামার সচেতনভাবে নির্দেশ করে যে এখানে আপাতত কোনো অবজেক্ট বা মান নেই।

JAVASCRIPT

EXPLICIT EMPTY

```
let selectedUser = null;  
console.log(selectedUser);
```

OUTPUT

```
null
```

22 `undefined` vs `null`

`undefined`

`null`

"আমি এখনো কোনো ভ্যালু পাইনি।" (ইঞ্জিনের অটোমেটিক ইনিশিয়াল মান)

"আমি ইচ্ছা করে বলছি এখানে কোনো ভ্যালু নেই।" (প্রোগ্রামারের সচেতন এসাইনমেন্ট)

```
let user;
```

```
let user = null;
```

23 `typeof null`

```
console.log(typeof null);
```

OUTPUT

```
object
```

⚠️ **ইন্টারভিউ প্রশ্ন:** `typeof null === "object"` জাভাস্ক্রিপ্টের প্রথম সংস্করণের একটি পরিচিত ঐতিহাসিক বাগ। ব্যাকওয়ার্ড কম্প্যাটিবিলিটির জন্য এটি ফিক্স করা হয়নি।

24 BigInt

Number টাইপ $2^{53} - 1$ এর বেশি বড় পূর্ণসংখ্যায় প্রিসিশন ধরে রাখতে পারে না। বিশাল পূর্ণসংখ্যা নির্ভুলভাবে সংরক্ষণের জন্য `BigInt` ব্যবহৃত হয়। সংখ্যার শেষে `n` দিলে BigInt literal তৈরি হয়:

```
const bigNumber = 123456789012345678901234567890n;  
console.log(bigNumber);
```

OUTPUT

```
123456789012345678901234567890n
```

25 BigInt Type & Restrictions

JAVASCRIPT

typeof BIGINT

```
const num = 12345678901234567890n;  
console.log(typeof num); // bigint  
  
const a = 10n;  
const b = 20n;  
console.log(a + b); // 30n
```

সতর্কতা: BigInt এবং সাধারণ Number সরাসরি পাটিগণিতে মিশ্রিত করা যায় না: `10n + 20` দিলে `TypeError: Cannot mix BigInt and other types` হবে!

26 Symbol

`Symbol` হলো একটি সম্পূর্ণ ইউনিক এবং ইমিউটেবল প্রিমিটিভ ভ্যালু, যা অবজেক্টের হিডেন বা ইউনিক প্রোপার্টি কি তৈরিতে ব্যবহৃত হয়:

```
const id = Symbol("id");  
console.log(id);
```

27 Symbols are Unique

JAVASCRIPT

SYMBOL UNIQUENESS

```
const a = Symbol("id");
const b = Symbol("id");

console.log(a === b);
```

OUTPUT

```
false
```

যদিও description একই ("id"), তবুও দুটো Symbol আলাদা।

সহজভাবে: `Symbol("id") ≠ Symbol("id")` (দুটো Symbol সম্পূর্ণ স্বতন্ত্র। এটা advanced JavaScript-এ অবজেক্ট প্রোপার্টি লুকিয়ে রাখতে কাজে লাগে)।

PART 03 — NON-PRIMITIVES: OBJECTS, ARRAYS & FUNCTIONS (28 – 34)

28 Object

Object হলো সম্পর্কিত ডেটা এবং ফাংশনালিটিকে Key-Value পেয়ার আকারে সংরক্ষণ করার কাঠামো:

JAVASCRIPT

OBJECT LITERAL

```
const student = {
  name: "Ripon",
  age: 22,
  department: "CSE"
};
```

29 Object Data Access

ডট নোটেশন (Dot notation) দিয়ে অবজেক্টের মান অ্যাক্সেস করা যায়:

JAVASCRIPT

ACCESSING PROPERTIES

```
const student = { name: "Ripon", age: 22 };
console.log(student.name);
console.log(student.age);
```

OUTPUT

```
Ripon
22
```

30 Array

Array হলো একাধিক value ordered collection হিসেবে রাখার জন্য ব্যবহৃত structure। প্রতিটি element-এর শূন্যভিত্তিক index থাকে:

JAVASCRIPT

ARRAY STRUCTURE

```
const fruits = ["Apple", "Mango", "Banana"];
console.log(fruits);
```

OUTPUT

```
["Apple", "Mango", "Banana"]
```

31 Array is an Object

জাভাস্ক্রিপ্টে Array আসলে বিশেষায়িত Object:

```
const fruits = ["Apple", "Mango"];
console.log(typeof fruits);
```

OUTPUT

object

32 Function

Function হলো পুনরায় ব্যবহারযোগ্য কোডের ব্লক। জাভাস্ক্রিপ্টে ফাংশন ফাস্ট-ক্লাস সিটিজেন:

JAVASCRIPT

FUNCTION

```
function greet() {  
    console.log("Hello!");  
}  
greet();  
console.log(typeof greet);
```

OUTPUT

```
Hello!  
function
```

33 Primitive Data Types — Full List

JAVASCRIPT

ALL 7 PRIMITIVES

```
const name = "Ripon";           // String
const age = 22;                  // Number
const huge = 123456789n;        // BigInt
const isStudent = true;         // Boolean
let result;                      // Undefined
const empty = null;             // Null
const id = Symbol("id");        // Symbol
```

34 `typeof` Complete Cheat Sheet

Value	typeof Output	মন্তব্য
"Hello"	"string"	Primitive string
100	"number"	Primitive numeric
10n	"bigint"	Arbitrary precision integer
true	"boolean"	Primitive boolean
undefined	"undefined"	Declared without value
null	"object"	⚠️ Historical language quirk
{}	"object"	Plain object literal
[]	"object"	Array is a specialized object
function(){}	"function"	Callable object

PART 04 — TYPE SYSTEM, DYNAMIC TYPING & COERCION (35 – 47)

35 Static vs Dynamic Typing

JavaScript একটি **dynamically typed language**। চলকের টাইপ আগে থেকে নির্ধারণ করতে হয় না, এবং একই ভ্যারিয়েবলে রানটাইমে বিভিন্ন টাইপের মান এসাইন করা যায়:

JAVASCRIPT

DYNAMIC REASSIGNMENT

```
let value = 100;
console.log(typeof value); // number

value = "Hello";
console.log(typeof value); // string
```

OUTPUT

```
number
string
```

36 JavaScript-এ Type Declaration

জাভাস্ক্রিপ্টে C/Java-এর মতো `int age = 22;` লিখতে হয় না। `let age = 22;` লিখলে ইঞ্জিন স্বয়ংক্রিয়ভাবে ভ্যালু দেখে ডেটা টাইপ বুঝে নেয়।

37 Dynamic Typing-এর Real-Life Example

JAVASCRIPT

E-COMMERCE STATE

```
let product = 500;
console.log(product);

product = "Out of Stock";
console.log(product);
```

OUTPUT

```
500
Out of Stock
```

38 Type Conversion

এক টাইপের ডেটাকে অন্য টাইপে রূপান্তর করাকে Type Conversion বলে: `String → Number`, `Number → String`, `Value → Boolean`।

39 String to Number

JAVASCRIPT

NUMBER() CONVERSION

```
const value = "100";
const number = Number(value);
console.log(number);
console.log(typeof number);
```

OUTPUT

```
100
number
```

40 Number to String

JAVASCRIPT

STRING() CONVERSION

```
const age = 22;
const text = String(age);
console.log(text);
console.log(typeof text);
```

OUTPUT

```
22
string
```

41 Boolean Conversion

JAVASCRIPT

BOOLEAN() CONVERSION

```
console.log(Boolean(1));  
console.log(Boolean(0));
```

OUTPUT

```
true  
false
```

42 Falsy Values

জাভাস্ক্রিপ্টে ৮টি মান বুলিয়ান কনটেক্সটে সর্বদা `false` দেয়:

`false`, `0`, `-0`, `0n`, `""` (খালি স্ট্রিং), `null`, `undefined`, `NaN`

```
console.log(Boolean(""));           // false  
console.log(Boolean("Hello"));      // true
```

43 Type Coercion

Type conversion এবং coercion-এর মধ্যে সূক্ষ্ম কিন্তু মৌলিক পার্থক্য রয়েছে:

TYPE CONVERSION (EXPLICIT – প্রোগ্রামার নিজে করে)

```
Number("100"); // 100 (Explicitly সংখ্যায় রূপান্তর)
```

TYPE COERCION (IMPLICIT – ইঞ্জিন নিজে করে)

```
console.log("10" + 5);
```

OUTPUT

```
105
```

কারণ এখানে জাভাস্ক্রিপ্ট কনটেক্সট বুঝে `5`-কে স্ট্রিং বানিয়ে কনক্যাট করেছে।

44 '+' Operator-এর Interesting Behavior

JAVASCRIPT

COERCION CONTRAST

```
// + অপারেটর স্ট্রিং পেলে কনক্যাট (যোগ) করে:  
console.log("10" + 5); // "105"  
  
// - অপারেটর কেবল সংখ্যা বোঝে, তাই "10"-কে Number বানিয়ে বিয়োগ করে:  
console.log("10" - 5); // 5
```

OUTPUT

```
105  
5
```

45 More Type Coercion Examples

JAVASCRIPT

MULTIPLICATION & DIVISION COERCION

```
console.log("5" * 2); // 10
console.log("10" / 2); // 5
console.log("10" - 3); // 7
console.log("10" + 3); // 103
```

OUTPUT

```
10
5
7
103
```

46 Explicit vs Implicit Conversion

EXPLICIT CONVERSION (DEVELOPER নিজে রূপান্তর করছে)

```
const age = Number("22");
```

IMPLICIT CONVERSION (JAVASCRIPT ENGINE নিজে রূপান্তর করছে)

```
console.log("22" - 2);
```

OUTPUT

```
20
```

47 Equality-এর Basic Introduction

JavaScript-এ দুই ধরনের equality operator আছে: `=` এবং `===`। এগুলো অত্যন্ত গুরুত্বপূর্ণ এবং পরবর্তী **Chapter 3 (Operators)**-এ বিস্তারিত আলোচনা করা হবে। নিচে এদের মৌলিক পার্থক্য তুলে ধরা হলো:

LOOSE EQUALITY `'=='` (TYPE COERCION করে)

```
console.log(5 == "5");
```

OUTPUT

```
true
```

STRICT EQUALITY `'==='` (VALUE ও TYPE দুটোই COMPARE করে)

```
console.log(5 === "5");
```

OUTPUT

```
false
```

Loose Equality `=`

তুলনার আগে স্বয়ংক্রিয় টাইপ রূপান্তর (Coercion) ঘটায়

`5 == "5" → true`

Strict Equality `===`

মান (Value) এবং টাইপ (Type) উভয়ই কঠোরভাবে পরীক্ষা করে

`5 === "5" → false`

★ **Beginner Rule:** Modern JavaScript-এ সাধারণত comparison-এর জন্য সর্বদা `===` ব্যবহার করা বেশি predictable ও নিরাপদ।

48 Primitive Values are Immutable

Primitive values-কে সাধারণভাবে immutable বলা হয়। এর মানে primitive value-কে সরাসরি modify করা যায় না; নতুন value তৈরি হয়।

JAVASCRIPT

IMMUTABILITY

```
let name = "Ripon";
name[0] = "X"; // String-এর character এভাবে modify হয় না
console.log(name);
```

OUTPUT

Ripon

যদি নতুন value দিতে চাও, তবে নতুন value assign করতে হবে:

```
name = "Xipon"; // এখানে নতুন String value assign হচ্ছে – আগের String পরিবর্তন হচ্ছে না
```

49 Primitive vs Reference — Copying Behavior

প্রিমিটিভে **Value Copy** হয়:

JAVASCRIPT

PASS BY VALUE

```
let a = 10;
let b = a;
b = 20;

console.log(a); // 10
console.log(b); // 20
```

OUTPUT10
20

50 Object Reference Example

অবজেক্টের ক্ষেত্রে ভ্যালু কপি হয় না, মেমোরি **Reference (ঠিকানা)** শেয়ার হয়:

JAVASCRIPT

PASS BY REFERENCE

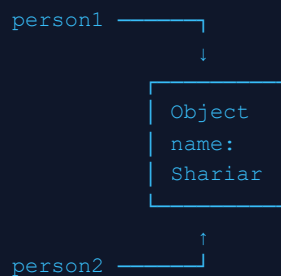
```
const person1 = { name: "Ripon" };
const person2 = person1;

person2.name = "Shariar"; // person2 দিয়ে পরিবর্তন করা হলো

console.log(person1.name);
console.log(person2.name);
```

OUTPUT

```
Shariar
Shariar
```



51 `const` Object নিয়ে Important বিষয়

অনেকে ভাবে: `const` হলে object-এর ভিতরের value change করা যাবে না। এটা পুরোপুরি ঠিক নয়।

JAVASCRIPT

MUTATING CONST OBJECT

```
const user = {
  name: "Ripon"
};

user.name = "Shariar"; // প্রোপার্টি পরিবর্তন করা সম্পূর্ণ বৈধ!
console.log(user.name);
```

OUTPUT

```
Shariar
```

কাজ করেছে। কিন্তু অবজেক্টের ভ্যারিয়েবল বাইন্ডিং রি-অ্যাসাইন করা যাবে না:

```
// user = {};
// ❌ TypeError: Assignment to constant variable!
// কারণ const binding-কে নতুন object-এ reassign করা যাবে না।
```

52 `const` Array-এর ক্ষেত্রেও একই ধারণা

JAVASCRIPT

MUTATING CONST ARRAY

```
const fruits = ["Apple", "Mango"];
fruits.push("Banana"); // সম্পূর্ণ বৈধ!
console.log(fruits);

// fruits = ["Orange"]; // TypeError: Assignment to constant variable!
```

OUTPUT

```
["Apple", "Mango", "Banana"]
```

53 Memory Concept — Basic Mental Model

Primitive:

a = 10 → [10] (স্বতন্ত্র কপি)

b = a → [10] (স্বতন্ত্র কপি)

Object:

person1 →
 └─→ [{ name: "Shariar" }] (একই মেমোরি রেফারেন্স)
person2 →

54 A Complete Example

এই অধ্যায়ের সমস্ত ডেটা টাইপের সমন্বয়ে একটি বাস্তব উদাহরণ:

JAVASCRIPT

SYNTHESIZED SUITE

```
const name = "Shariar";
let age = 22;
const cgpa = 3.75;
const isStudent = true;
let university;
const address = null;
const skills = ["HTML", "CSS", "JavaScript"];
const student = { name, age, cgpa };

console.log(name, age, cgpa, isStudent, university, address);
console.log(skills);
console.log(student);
```

OUTPUT

```
Shariar 22 3.75 true undefined null
["HTML", "CSS", "JavaScript"]
{ name: 'Shariar', age: 22, cgpa: 3.75 }
```

55 Data Types Cheat Sheet

Data Type	Example	typeof Output
String	"Hello"	"string"
Number	100	"number"
BigInt	100n	"bigint"
Boolean	true	"boolean"
Undefined	undefined	"undefined"
Null	null	"object" *
Symbol	Symbol("id")	"symbol"
Object	{name: "Ripon"}	"object"
Array	[1, 2, 3]	"object"
Function	function(){}	"function"

Chapter 2 — Core Concepts You Must Remember

1 Data: Program যে information নিয়ে কাজ করে।

2 Data Type: Value-এর ধরন ও অপারেশন নির্দেশ করে।

3 Primitive: ৭টি ইমিউটেবল বেসিক মান (String, Number, BigInt, Boolean, Undefined, Null, Symbol)।

4 Non-Primitive: Object, Array, Function (Reference based)।

5 Dynamic Typing: রানটাইমে ভ্যালু অনুযায়ী টাইপ পরিবর্তিত হয়।

6 Type Conversion: ডেভেলপার নিজে রূপান্তর করে (Explicit)।

7 Type Coercion: ইঞ্জিন স্বয়ংক্রিয়ভাবে রূপান্তর করে (Implicit)।

8 `typeof`: রানটাইমে টাইপ চেক করার অপারেটর।

9 `==` vs `===`: Loose (coercion সহ) বনাম Strict Equality।

10 Primitive vs Object: Value Copy বনাম Memory Reference Share।

Chapter 2 — Practice Problems

Practice 1:

```
const name = "Ripon";
const age = 22;
const cgpa = 3.75;
const isStudent = true;

console.log(typeof name, typeof age, typeof cgpa, typeof isStudent);
```

আউটপুট কী হবে? (Expected: `string number number boolean`)

Practice 2 (Coercion Prediction):

```
console.log("10" + 5);
console.log("10" - 5);
console.log("10" * 2);
console.log("10" / 2);
```

আউটপুট প্রেডিক্ট করো। (Expected: `"105", 5, 20, 5`)

Practice 3 (Boolean Values):

```
console.log(Boolean(0));
console.log(Boolean(1));
console.log(Boolean(""));
console.log(Boolean("JavaScript"));
console.log(Boolean(null));
console.log(Boolean(undefined));
```

Practice 4 (Dynamic Typing):

```
let value = 100;
console.log(typeof value);
value = "JavaScript";
console.log(typeof value);
value = true;
console.log(typeof value);
```

Practice 5 (Object Reference Trap):

```
const user1 = { name: "Ripon" };
const user2 = user1;
user2.name = "Shariar";
console.log(user1.name); // আউটপুট কেন "Shariar" হবে তা ব্যাখ্যা করো!
```

🚀 Mini Project — Student Data Profile

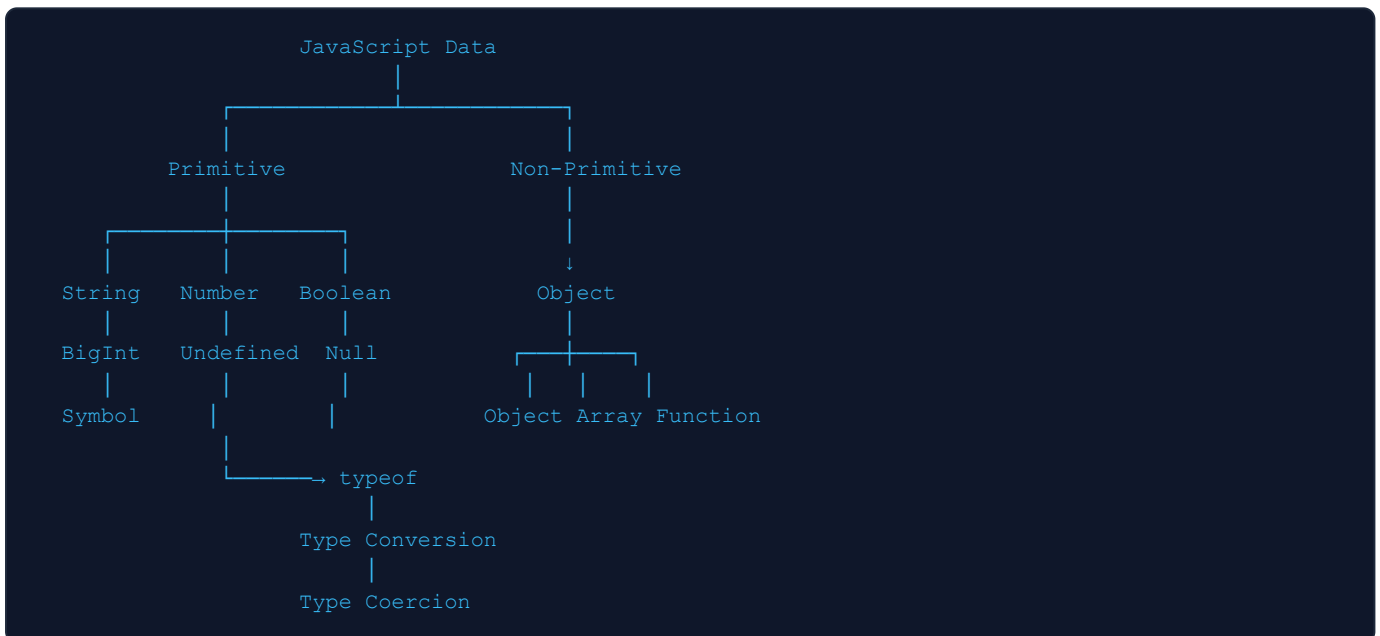
JAVASCRIPT

STUDENT PROFILE SCRIPT

```
const name = "Shariar";
const age = 22;
const studentId = "231-15-010";
const cgpa = 3.75;
const isStudent = true;
const skills = ["HTML", "CSS", "JavaScript"];
const address = { city: "Dhaka", country: "Bangladesh" };

console.log("Name:", name);
console.log("Age:", age);
console.log("Student ID:", studentId);
console.log("CGPA:", cgpa);
console.log("Student:", isStudent);
console.log("Skills:", skills);
console.log("Address:", address);
```

🚀 Chapter 2 Final Mental Map



Chapter 2-এর core foundation এখানেই। এর পরের logical step হলো Chapter 3 — Operators & Expressions, যেখানে `+`, `-`, `*`, `/`, `%`, `**`, assignment, comparison, logical, ternary, nullish coalescing এবং optional chaining বিস্তারিত শেখা হবে।